



First Hiss

Your first quick-and-dirty crash course in Python

Nimish Chandiramani

Disclaimer: This tutorial covers the very basics of Python programming. You'll need some basic programming knowledge for this. If you're already familiar with the language, you could consider soaking in the other wonderful articles in this issue.

If you're a programmer, or want to learn programming, you're going to love Python. It's one of the easiest languages you'll ever learn, it's powerful, and best of all, you can use it to build cross-platform applications.

Get Python

If you're a Linux or Mac user, you've already got Python installed. However, you should still download the latest version — we're using Python 3, which has made a few changes to the syntax. You can get Python for your platform from www.python.org. The installation process is pretty straightforward.

The First Time

There are two ways you can work with Python — you could either write your programs in a regular text editor and run them in the Python environment, or type your commands directly into the interpreter. If you're old enough to remember BASIC, you're already familiar with this. For now, let's work with the interpreter. You have two options: if you want to see your code in pretty colours, start IDLE — the *integrated development environment* (IDE) that installs with Python; or if you prefer the geeky goodness of the command line, fire up your terminal (Start > Run > type "cmd" in Windows) and type "python".

Both environments are rather uneventful-looking, and all you'll see is a prompt that looks like this: ">>>".

And now, you're ready to start with the world's simplest program. In the prompt, type "print('Hello, world!')" (you can use single or double quotes — the interpreter isn't fussy). And that's it! Your first program: no messy surrounding code — just a command. You'll notice that Python is consistently like this — it's practically like having a conversation with your PC.

```
***code***
>>> print('Hello, world!')
Hello, world!
***code***
```

Caption: Your first Python command in action

Know the rules

Before you move to the more complex world of two-line programs, you must first familiarise yourself with Python's basic syntax. If you've had some experience with any other programming language, this can be a little disconcerting. Firstly, there's no end-of-line character. If you hit enter, you've ended a line — simple. Second, there are no beginning-and-end braces, brackets or anything — a program's blocks are denoted only by indentation.

Other programming languages want you to declare variables before you use them in your program. Python is very forgiving in this matter. To start using a new variable in your program, all you need to do is give it a value — the Python interpreter will automatically decide what type of variable it is, and give it its own special place in your system's memory. Let's look at a program that calculates the area of a circle, and how you'll see it in the interpreter:

```
***code***
>>>pi=3.14159          # <- this hash means
```

you're starting a comment

```
>>>r=4                  # the radius of the
circle
>>>area=pi*(r**2)      # remember that even
Python follows the BODMAS rule — brackets are always
solved first
>>>print(area)
50.25644
***code***
```

Getting Modular

Once you're ready to start writing multiple-line programs, you're better off doing so in a Python file — called a *module*. Once you've written a module, you can execute it from the command line whenever you feel like it. There's also a more significant use for modules, but we'll come to that in just a bit.

You can create modules in any text editor (see *Kit Bag* for our favourites). If you're using IDLE, all you need to do is hit [Ctrl] + [N] to open a new window. When you're ready, start typing. Let's use the same example — calculating the area of a circle:

```
***code***
# Calculate the area of the circle
pi = 3.14159
r = 4
area = pi*(r**2)        # '***' is the python
equivalent of 'to the power of'
print('The area is ',area)
***code***
```

Save this file as a .py file on your hard disk. If you're using IDLE, you just need to hit [F5] to run the module. If you've used a text editor, open your terminal (command prompt in Windows), navigate to where you stored your file, and type "python filename.py".

But a program that calculates the area of just one circle isn't really fun, is it?

Getting Interactive

To make this program slightly more useful, let's have it ask the user for a radius. We do this by using the input() function, which will (as the name suggests), accept an input from the user.

```
***code***
# Calculating the area of a circle, v2

pi = 3.14159
r = input('What is the radius? ')      # the
question is optional
area = pi*(r**2)
print('The area is ',area)
***code***
```

When you run this file, you'll see this:

```
***code***
$ python area.py
What is the radius? 5
The area is 78.53975
***code***
```

Now we're getting somewhere. This would be a good time to fiddle with some conditional statements.

```
***code***
```